On Efficient Computations of $y^2 = x^3 + b/\mathbb{F}_p$ for Primes $p \equiv 1 \pmod 3$

Guangwu Xu[*], Wei Yu[†], Ke Han[‡], Pengfei Lu[§]

Abstract

Solinas' window τNAF algorithm has been a very powerful scalar multiplication method designed for the class of Koblitz curves over binary fields. The idea is to use the Frobenius endomorphism τ which is a notable efficiently computable endomorphism (viewed as multiplication by complex algebraic integer). The method consists of two major steps: the first is a reduction of a scalar k to the form of $k_1 + k_2\tau$, with k_1, k_2 being of half of the size of the group order; the second is to use τ-adic non-adjacent form of $k_1 + k_2\tau$ to evaluate kP, where an efficient pre-computation is enabled based on a carefully chosen set of coefficients. It would desirable to develop a similar method for a large family of curves, especially for those practical curves over prime fields. In this effort, the GLV method was proposed as a suboptimal solution whose essential part is to create a reduction with respect to some efficiently computable endomorphism, and then use simultaneous multiplication method to evaluate kP. The GLV method has been the state-of-the-art for scalar multiplication for the family of curves $E_b : y^2 = x^3 + b/\mathbb{F}_p$ for primes $p \equiv 1 \pmod 3$. This family of curves is of practical significance, e.g., some of its instances are used in Blockchain platforms such as Bitcoin and Ethereum. The curves $E_b/\mathbb{F}_p$ have the ring $\mathbb{Z}[\omega]$ of Eisenstein integers as their ring of endomorphisms, and the scalar multiplication by the unit group $U \subset \mathbb{Z}[\omega]$ is especially efficient as ω corresponds to the map $(x, y) \mapsto (\beta x, y)$ (and $\mathcal{O} \mapsto \mathcal{O}$) where β is a cubic root of unity in $\mathbb{F}_p$. In this paper, we are able to settle a window τNAF algorithm for the class of curves $E_b/\mathbb{F}_p$, using the endomorphism $\tau = 1 - \omega$. We first derives several efficient formulas for some special scalars. More precisely, let $P = (x, y) \in E_b(\mathbb{F}_p)$, we have

$$\tau P = \left(\frac{x^3 + 4b}{(1-\beta)^2 x^2}, y \frac{x^3 - 8b}{(1-\beta)^3 x^3} \right).$$

[*]SCST, Shandong University, China, e-mail: gxu4sdq@sdu.edu.cn(Corresponding author)
[†]IIE, Chinese Academy of Sciences, China, e-mail: yuwei@iie.ac.cn(Corresponding author)
[‡]SCST, Shandong University, China, e-mail: 202237084@mail.sdu.edu.cn
[§]SCST, Shandong University, China, e-mail: PengfeiLu@mail.sdu.edu.cn

Converted to Jacobian projective coordinates, this operation of τ requires $6\mathbf{M}$ ($\mathbf{M}$ denotes the cost for a field multiplication) which is less expensive than a point doubling. The efficient formula of τP can be further used to speed up the computation of $3P$, compared to the current record of $15\mathbf{M}$, our tripling formula just costs $10\mathbf{M}$. We also propose efficient computations of $(2-\omega)P$ and $(3-\omega)P$. As $N(\tau)=3$, the fast computation of τ and nice properties of Eisenstein integers enable us to develop a window τNAF method for curves $E_b/\mathbb{F}_p$. This result is not only of theoretical interest but also of a higher performance due to the facts that (1) the window τNAF can be further turned to a sparse form that uses only tripling operation (which costs $10\mathbf{M}$) and augmented-τ operation (which costs $5\mathbf{M}$); (2) a pre-computation is carefully designed by choosing a set of coefficients that is U-invariant, such a symmetry simplifies the pre-computation in that only about one-sixth of the coefficients need to be processed. With respect to the number of field multiplication used, our optimized algorithm achieves more than $16.7\%, 17.6\%$ and 18% of improvement over the current state-of-the-art, for group orders of 256-bit, 384-bit, and 512-bit respectively. This paper also explores a regular window τNAF as a countermeasure against side-channel attacks. This regularized version significantly outperforms the regular GLV method, cutting costs by $17.7\%, 19.5\%$ and 20.9%, for group orders of 256-bit, 384-bit, and 512-bit respectively.

1 Introduction

Let $p \equiv 1 \pmod 3$ be a prime. We consider fast computations for the family of elliptic curves

$$E_b : y^2 = x^3 + b/\mathbb{F}_p,\ b \in \mathbb{F}_p^*. \tag{1}$$

This family of curves, also known as Koblitz curves over prime field $\mathbb{F}_p$, has a significant practical importance. In the standard for efficient cryptography [34], three curves of this sort are included: secp192k1, secp224k1 and secp256k1. In particular, the curve secp256k1

$$\text{secp256k1:}\quad y^2 = x^3 + 7/\mathbb{F}_p,$$

where $p = 2^{256} - 2^{32} - 977$ is a prime of 256 bits, is an allowed curve by the NIST recommendations for discrete logarithm-based cryptography [33]. The BLS pairing-friendly curves (including BLS12-381) are also in this family [1, 7]. These curves have been used for blockchain-related applications (e.g., digital signatures for Bitcoin, Ethereum and Zcash).

Let $\omega = \frac{-1+\sqrt{-3}}{2}$. We note that E_b is an elliptic curve with complex multiplication by $\mathbb{Z}[\omega]$, the ring of Eisenstein integers. In particular, ω corresponds to an efficiently-computable endomorphism Φ of E_b defined over $\mathbb{F}_p$. Let $\beta \in \mathbb{F}_p$ be a primitive third

root of unity, the map $\Phi : E_b \to E_b$ is given by $(x, y) \mapsto (\beta x, y)$ and $\mathcal{O} \mapsto \mathcal{O}$. The computation of $\Phi(x, y)$ only costs one multiplication in $\mathbb{F}_p$. This means that the action of the unit group $U = \{1, -1, \omega, -\omega, \omega^2, -\omega^2\} \subset \mathbb{Z}[\omega]$ on $E_b(\mathbb{F}_p)$ is computationally efficient. It is noted for this case, there is an easy and elementary way to compute a prime $\pi = c + d\omega \in \mathbb{Z}[\omega]$ such that $p = N(\pi)$ and $\#(\mathbb{F}_p) = N(\pi - 1)$ [25, 31]. These provides sufficient ingredients to compute a scalar product using the GLV method [13] in the group $E_b(\mathbb{F}_p)$. Indeed, E_b has been a typical subject in the study of GLV application and optimization which makes GLV the state-of-the-art for scalar multiplication of $E_b(\mathbb{F}_p)$, see, for example, [8, 9, 13, 15, 26].

Binary Koblitz curves are elliptic curves over $\mathbb{F}_2$ defined as

$$E_a : y^2 + xy = x^3 + ax^2 + 1/\mathbb{F}_{2^m},\ a \in \mathbb{F}_2. \tag{2}$$

These curves were proposed by Koblitz for cryptography because of their computational efficiency [17]. This family of curves provides a well known example of using an efficiently computable endomorphism in scalar multiplication. In this case, the Frobenius endomorphism $\tau_2 : E_a \to E_a$ given by $\tau_2(x, y) = (x^2, y^2)$ and $\tau_2(\mathcal{O}) = \mathcal{O}$ can be easily computed. The map τ_2 is regarded as a complex number satisfying $\tau_2^2 - (-1)^{1-a}\tau_2 + 2 = 0$ and $\mathbb{Z}[\tau_2]$ is a endomorphism ring of E_a.

The idea of Koblitz was substantially developed to the celebrated window τ_2NAF method (window τ_2-adic non-adjacent form) by Solinas in [28]. The main ingredients of width-w window τ_2NAF are reduction, sparse τ_2-expansion and pre-computation. Solinas devises a reduction procedure that converts an integer k to an element $k_1 + k_2\tau_2 \in \mathbb{Z}[\tau_2]$ such that $kP = (k_1 + k_2\tau_2)P$ and the sizes of k_1 and k_2 are about $\frac{m}{2}$. A window τ_2-adic non-adjacent form (τ_2NAF) of $k_1 + k_2\tau_2$ is then derived on a pre-selected set of coefficients $C = \{c_1, c_3, \ldots, c_{2^{w-1}-1}\}$ with $c_i \equiv i \pmod{\tau_2^w}$, which has the following sparse form

$$k_1 + k_2\tau_2 = \sum_{i=0}^{l-1} \varepsilon_i u_i \tau_2^i,$$

where $\varepsilon_i \in \{-1, 1\}$ and $u_i \in C \cup \{0\}$ with the property that any set $\{u_k, u_{k+1}, \ldots, u_{k+w-1}\}$ contains at most one nonzero element. This implies that for a point P,

$$kP = k_1P + k_2\tau_2(P) = \sum_{i=0}^{l-1} \epsilon_i u_i \tau_2^i(P) = \sum_{i=0}^{l-1} \epsilon_i \tau_2^i(u_i P).$$

The pre-computation produces $c_1P, c_3P, \cdots, c_{2^{w-1}-1}P$ once the set C is given, so u_iP's in the above evaluation are already available. Several sets of coefficients for pre-computation were reported in [28, 14]. It should be noted that determining a set of coefficients (or digits) of number representation with respect to a radix which is an algebraic integer is more complicated than that for a rational integer. This

issue for Solina's window τ_2NAF form is addressed by Blake, Murty and Xu in [4], where a theoretical framework for (sparse) τ_2-adic representation of integers in the Euclidean imaginary quadratic field $\mathbb{Q}(\tau_2)$ is developed. Further generations to (all) Euclidean imaginary quadratic fields are given in [5, 6]. The framework provides greater flexibility for choosing sets of coefficients which enables several efficient precomputations for window τ_2NAF for Koblitz curves [30, 32].

It would be desirable to extend Solinas' technique to a large family of curves, especially for those practical curves over prime fields. However according to Gallant, Lambert and Vanstone [13], in order to design an efficient window ΦNAF method based on a (complex) endomorphism Φ, some requirements about Φ have to be satisfied: (1) its computation is not more expensive than a point doubling; (2) the Φ-adic expansion of the scalar is not significantly longer than its binary expansion; and (3) $N(\Phi) > 1$. Since such an endomorphism was not available for curves over prime fields, the GLV method was proposed as a suboptimal solution in [13]. The essential part of GLV method is to create a reduction with respect to some efficiently computable endomorphism, and then use simultaneous multiplication method to evaluate kP. As mentioned earlier, for the specific example of $E_b : y^2 = x^3 + b/\mathbb{F}_p$, the efficient endomorphism ω renders the GLV method the fastest known technique for scalar multiplication.

The main purpose of the paper is to settle the problem of establishing a window τNAF algorithm for the family of $E_b/\mathbb{F}_p$. As the first step, we derive efficient formulas for several endomorphisms for $E_b/\mathbb{F}_p$, some of them meets the requirements for developing window τNAF to perform efficient scalar multiplication in $E_b(\mathbb{F}_p)$. Other formulas also contribute efficiency of elliptic curve operation in some ways.

To be more specific, we obtain efficient expressions for the endomorphisms $1-\omega, 2-\omega$ and $3-\omega$ in the first part of the paper. It is interesting to note that these three numbers are prime factors of the first three rational primes (namely, $3, 7$ and 13) that are factorizable in $\mathbb{Z}[\omega]$. We write $\tau = 1-\omega$. For $P = (x, y) \in E_b(\mathbb{F}_p)$, our formula for τP is

$$\tau P = \left(\frac{x^3 + 4b}{(1-\beta)^2 x^2}, y \frac{x^3 - 8b}{(1-\beta)^3 x^3} \right). \tag{3}$$

This expression makes the conversion to Jacobian projective coordinates more convenient. Under Jacobian projective form, the computation of τP requires $3\mathbf{S} + 3\mathbf{M}$ ($\mathbf{S}$ and $\mathbf{M}$ denote the costs for field squaring and multiplication, respectively), which is less expensive than a point doubling. Note that $3 = N(\tau) = (1-\omega)(1-\omega^2)$, a computation of tripling (i.e., $3P$) can be designed based on (3) by using $4\mathbf{S} + 6\mathbf{M}$. According to the "Explicit-Formulas Database" [3] (for the case of Short Weierstrass curves with $a_4 = 0$ and Jacobian coordinates), the previous best operation count for tripling is $10\mathbf{S} + 5\mathbf{M}$ or $7\mathbf{S} + 8\mathbf{M}$ for $E_b/\mathbb{F}_p$. The computation of $3P$ can be used in a optimized version of our window τNAF. The formula is also of independent interest in

that $3P$ is a main ingredient for double base chain method for scalar multiplication, a greater efficiency can be achieved by using the new formula.

Our tripling formula for $E_b/\mathbb{F}_p$ is in fact a composition of two endomorphisms (isogenies): $3 = \tau\overline{\tau}$, with some optimization. It is interesting to note that Doche, Icart, and Kohel propose a tripling formula for curve $y^2 = x^3 + u(x^2+1)^3$ over a finite field of odd characteristic in [11]. Their scalar multiplication map-by-3 is obtained as the composition of two isogenies. With the new Jacobian coordinates, the cost of computation can be $6\mathbf{S} + 6\mathbf{M}$ if the parameter of the curve is suitably chosen.

The formulas of $2-\omega$ and $3-\omega$ are more complicated, but their expressions still display some useful computational features and translate to a better performance. For example in Jacobian projective coordinates, the computation of $(2-\omega)P$ costs $4\mathbf{S} + 9\mathbf{M}$ which is less expensive than a point addition. As $7 = N(2-\omega)$, we see that $7P$ can be computed using $8\mathbf{S} + 18\mathbf{M}$.

In the second part of the paper, we describe a window τNAF method for the scalar multiplication in $E_b(\mathbb{F}_p)$ based on our efficient calculation of the endomorphism τ.

The window τNAF method begins with the reduction procedure that turns a scalar k to a form of $k_1 + k_2\tau$ with small k_1, k_2. Our reduction is a refined version of the reduction for the case of E_b proposed by Brown, Myers and Solinas in [8]. Note that the reduction in [8] was designed for GLV method and the approach is similar to the reduction procedure of Solinas in [28], where the Voronoi cell is used to get an optimal bound. It is remarked that a reduction technique was proposed in [13] under the term of scalar decomposition which is obtained from short vectors of certain 2-dimensional lattice. Some further studies on improving scalar decomposition can be found in [8, 15, 26, 27].

The set of coefficients C for our window τNAF is carefully designed, some principles described in [5, 6] are based. To minimize the cost of pre-computation, we select the set $C \subset \mathbb{Z}[\omega]$ to be U-invariant:

$$UC = C,$$

namely, for any $c \in C$, $-c, \pm\omega c, \pm\omega^2 c$ are all in C. By this U-invariant property, only one-sixth of the coefficients need to be processed for pre-computation and the rest can be obtained by multiplying units $\{-1, \pm\omega, \pm\omega^2\}$ of $\mathbb{Z}[\omega]$.

In this paper, we consider window τNAF with width $w = 4$. In this case, a window τNAF form for $a + b\tau$ is

$$a + b\tau = c_0\tau^{\ell_0} + c_1\tau^{\ell_1} + \cdots + c_{t-1}\tau^{\ell_{t-1}}$$

where the non-zero coefficient $c_j \in C$ and $\ell_j - \ell_{j-1} \geq 4$. By the U-invariant property and the fact that $\tau^2 = -3\omega$, the above form can be turned to

$$a + b\tau = c'_0\tau^{\delta_0}3^{\lfloor\frac{\ell_0}{2}\rfloor} + c'_1\tau^{\delta_1}3^{\lfloor\frac{\ell_1}{2}\rfloor} + \cdots + c'_{t-1}\tau^{\delta_{t-1}}3^{\lfloor\frac{\ell_{t-1}}{2}\rfloor}, \tag{4}$$

where $c'_j = c_j(-\omega)^{\lfloor \frac{\ell_j}{2} \rfloor} \in C$ and $\delta_j \in \{0, 1\}$. We shall explain later, the U-invariant property gives us some extra information that reduces the calculation cost of τP to $2\mathbf{S} + 3\mathbf{M}$. Note that the cost for $3P$ is $4\mathbf{S} + 6\mathbf{M}$, we can use (4) to get an optimized scalar multiplication. Together with a refined pre-computation, our algorithm achieves a significant of improvement over the current state-of-the-art. With respect to the number of field multiplication used, for groups of order 256-bit, 384-bit, and 512-bit, the improvements are more than $16.7\%, 17.6\%$ and 18% respectively. The paper also contains a regular window τNAF for the purpose of protecting scalar multiplication against simple side-channel attacks. This regularized version significantly outperforms the regular GLV method, cutting costs by $17.7\%, 19.5\%$ and 20.9%, for group orders of 256-bit, 384-bit, and 512-bit respectively.

A generalization of binary window non-adjacent form (for radix 2) is proposed in [29] to window ℓ-NAF method with radix ℓ, for an integer $\ell \geq 2$. More efficient way of compute the multiplication-by-ℓ map using simple isogeny decomposition is investigated in [11] and is used in window ℓ-NAF for speeding up scalar multiplication. We remark that the window ℓ-NAF method and our window τNAF are different in several ways: (1) ℓ is a rational integer, while τ is an integer in a Euclidean imaginary quadratic number field; (2) the cost of multiplication-by-τ is much less than that of multiplication-by-ℓ; (3) while pre-computation of window ℓ-NAF is quite simple and efficient, pre-computation of window τNAF appears to be more flexible so one can choose suitable coefficients for τ-expansion for efficiency, one can also perform regular recoding using such property. In general, if applicable, the GLV method is superior to the ℓ-NAF method. For a subgroup $G \subset E_b(\mathbb{F}_p)$ of size 256-bit, even we could use our efficient $3P$ formula in the window 3-NAF routine (with a window size being 3), its cost would be more than $2400\mathbf{M}$, but the cost of GLV is under $1800\mathbf{M}$. Since our window τNAF improves GLV, so it is better than ℓ-NAF for $E_b/\mathbb{F}_p$.

The rest of our paper is arranged into four sections. Section 2 provides some preliminaries and develops some tools. The derivation of efficient formulas is described in section 3. The main algorithms and some discussions are given in the section 4. We conclude the paper in section 5.

2 Preliminaries

The curves we discuss in the paper are of the form (1):

$$E_b : y^2 = x^3 + b/\mathbb{F}_p,$$

where the prime $p \equiv 1 \pmod 3$ and $b \in \mathbb{F}_p^*$. Modulo such a prime p, there is a primitive cubic root of unity β, i.e., $\beta \neq 1$ and $\beta^3 \equiv 1 \pmod p$.

According to a refinement of the point counting formula of Rajwade [25] reported in [31], one can efficiently calculate a prime $\pi = c' + d'\omega \in \mathbb{Z}[\omega]$ such that $p = N(\pi)$ and $\#E(\mathbb{F}_b) = N(\pi - 1)$.

Suppose $\#E_b(\mathbb{F}_p) = hn$ for some prime number n and some small cofactor h. In practical applications, it is often necessary to require that $n \equiv 1 \pmod 3$. In this case, it is easy to get integers c, d such that $N(c + d\omega) = n$, namely

$$n = c^2 - cd + d^2. \tag{5}$$

We shall assume $n \equiv 1 \pmod 3$ with $n = c^2 - cd + d^2$ in the rest of the paper. Let $G \subset E_b(\mathbb{F}_p)$ be a subgroup of order n

For $P = (x, y) \in E_b(\mathbb{F}_p) \setminus \{\mathcal{O}\}$, the endomorphism Φ is given by $\Phi(P) = \omega P = (\beta x, y)$. We also have $\Phi^2(P) = \omega^2 P = (\beta^2 x, y)$. Since G is cyclic, Φ acts on it as a scalar multiplication by an integer $\lambda < n$. So for any point $(x, y) \in G \setminus \{\mathcal{O}\}$,

$$\lambda(x, y) = (\beta x, y).$$

It is easy to conclude that λ is a primitive cube root of unity modulo n. Further more, we have $\lambda^2(x, y) = (\beta^2 x, y)$.

Scalar reduction: The purpose of the scalar reduction (aka scalar decomposition) is to compute the following decomposition of a scalar $k < n$

$$k \equiv k_1 + k_2\lambda \pmod n \tag{6}$$

such that the integers k_1 and k_2 are both small (on the magnitude of $\sqrt{n}$). As mentioned earlier, there are several proposals to optimize the decomposition with different ways. We shall adopt the one given in [8] (see, also [15]). The procedure starts by computing $x = \frac{k(c-d)}{n}$ and $y = \frac{-kd}{n}$, and then setting

$$q_1 = \left\lfloor \frac{\lfloor x + y \rfloor + \lfloor 2x - y \rfloor + 2}{3} \right\rfloor, \; q_2 = \left\lfloor \frac{\lfloor x + y \rfloor + \lfloor 2y - x \rfloor + 2}{3} \right\rfloor. \tag{7}$$

It can be seen that

$$\begin{aligned} k_1 &= k - q_1 c + q_2 d, \\ k_2 &= q_2 d - q_1 d - q_2 c \end{aligned} \tag{8}$$

gives the desired relation

$$k \equiv k_1 + k_2\lambda \pmod n.$$

and $|k_1|, |k_2| < \frac{2\sqrt{n}}{3}$. It is noted that $\frac{2\sqrt{n}}{3}$ is an optimal bound [15].

Note that for every k, divisions are performed in computing x, y. We propose a simplification of the calculation by using a one-time pre-computation (once for the curve) so that the divisions can be removed. To this end, we let

$$s = \lceil \log_2 n \rceil,\ t = \lceil \log_2 2(|c| + |d|) \rceil,$$

and pre-compute three constants

$$\eta_1 = \left\lfloor \frac{(c-2d)2^{s+t}}{n} \right\rfloor,\ \eta_2 = \left\lfloor \frac{(2c-d)2^{s+t}}{n} \right\rfloor,\ \eta_3 = \left\lfloor \frac{(-c-d)2^{s+t}}{n} \right\rfloor.$$

Then for a given $0 \leq k < n$, we calculate $f_j = \left\lfloor \frac{k\eta_j}{2^{s+t}} \right\rfloor,\ j = 1, 2, 3$, and get q_1, q_2 as follows:

$$q_1 = \left\lfloor \frac{f_1 + f_2 + 2}{3} \right\rfloor,\ q_2 = \left\lfloor \frac{f_1 + f_3 + 2}{3} \right\rfloor. \tag{9}$$

Putting this pair of (q_1, q_2) into (8), we get the decomposition (6) of k in terms of λ. It is straightforward to get a decomposition of k in terms of $\tau = 1 - \lambda$:

$$k \equiv k_1 + k_2\lambda \pmod{n} = (k_1 + k_2) + (-k_2)\tau \pmod{n}. \tag{10}$$

We make some remarks on the simplified procedure.

Remarks

1. It is seen that we need to compute 3 multiplications $k\eta_1, k\eta_2, k\eta_3$. The division by 2^{s+t} is trivial.

2. The integers q_1, q_2 are close to x, y respectively. For example, it is easy to verify that
$$3x - \frac{1}{2^{t-1}} \leq f_1 + f_2 + 2 < 3x + 2.$$

Jacobian projective coordinates: For every $(X, Y, Z) \in \mathbb{F}_p^3$, Jacobian projective coordinates $(X : Y : Z)$ is the following equivalent class defined over $\mathbb{F}_p^3$

$$(X : Y : Z) = \{(\lambda^2 X, \lambda^3 Y, \lambda Z) : \lambda \in \mathbb{F}_p^*\}.$$

If $Z \neq 0$, Jacobian projective coordinates $(X : Y : Z)$ corresponds to the affine coordinate $(\frac{X}{Z^2}, \frac{Y}{Z^3})$. The equation of $E_b/\mathbb{F}_p$ under the Jacobian projective coordinates becomes

$$E_b : Y^2 = X^3 + bZ^6, \tag{11}$$

with $(1 : 1 : 0)$ being the point of infinity.

Under projective coordinates, if two non-trivial points of $E(\mathbb{F}_p)$

$$P = (X_P : Y_P : Z_P), Q = (X_Q : Y_Q : Z_Q)$$

satisfy $P \neq -Q$ and if their sum is $R = (X_R : Y_R : Z_R)$, then putting $\left(\frac{X_P}{Z_P^2}, \frac{Y_P}{Z_P^3}\right), \left(\frac{X_Q}{Z_Q^2}, \frac{Y_Q}{Z_Q^3}\right)$ in the addition formula in affine coordinate,

1. if $P \neq Q$, we have $\begin{cases} X_R = (Y_Q Z_P^3 - Y_P Z_Q^3)^2 \\ \qquad -(X_Q Z_P^2 - X_P Z_Q^2)^2 (X_P Z_Q^2 + X_Q Z_P^2), \\ Y_R = (Y_Q Z_P^3 - Y_P Z_Q^3)\big(X_P Z_Q^2 (X_Q Z_P^2 - X_P Z_Q^2)^2 - X_R\big) \\ \qquad -Y_P Z_Q^3 (X_Q Z_P^2 - X_P Z_Q^2)^3, \\ Z_R = (X_Q Z_P^2 - X_P Z_Q^2) Z_P Z_Q. \end{cases}$

2. if $P = Q$, we have $\begin{cases} X_R = 9X_P^4 - 8X_P Y_P^2, \\ Y_R = 3X_P^2(4X_P Y_P^2 - X_R) - 8Y_P^4, \\ Z_R = 2Y_P Z_P. \end{cases}$

In Jacobian projective coordinates, point addition and doubling for curves $E_b/\mathbb{F}_p$ can be performed by using the procedures in [14] (see also [3]). So the cost for addition in Jacobian coordinates is $4\mathbf{S} + 12\mathbf{M}$, and the cost for doubling is $4\mathbf{S} + 3\mathbf{M}$. In [20], point addition is improved by trading a multiplication with a squaring. The costs for point addition and point doubling for curves $E_b/\mathbb{F}_p$, as described in table 1, are

$$1\mathbf{ADD} = 5\mathbf{S} + 11\mathbf{M},\ 1\mathbf{DBL} = 5\mathbf{S} + 2\mathbf{M}.$$

Table 1: Point Addition and Doubling ($y^2 = x^3 + b$, Jacobian coordinates)

Addition Procedure of E	**Doubling Procedure of** E
$R = P + Q\ (P \neq \pm Q)$	$R = 2P$
$A = X_P Z_Q^2 - X_Q Z_P^2$;	$A = 3X_P^2$;
$B = Y_P Z_Q^3 - Y_Q Z_Q^3$;	$B = 2Y_P$;
$C = 2X_Q Z_P^2 A^2$;	$C = B^2$;
$D = 4Y_Q Z_P^3 A^3$	$D = CX_P = \frac{(X_P+C)^2 - X^2 - C^2}{2}$;
$X_R = 4(B^2 - A^3 - C)$;	$X_R = A^2 - 2D$;
$Y_R = 2(B(2C - X_R) - D)$;	$Y_R = (D - X_R)A - \frac{C^2}{2}$;
$Z_R = ((A+Z_Q)^2 - A^2 - Z_Q^2)Z_P$;	$Z_R = BZ_P$;

In this paper, we shall adopt the conversion $1\mathbf{S} = 1\mathbf{M}$[1]. This means that

$$1\mathbf{ADD} = 16\mathbf{M},\ 1\mathbf{DBL} = 7\mathbf{M}.$$

[1]It is remarked that one can use $1\mathbf{S} = 0.67\mathbf{M}$ or $0.8\mathbf{M}$ if side-channel attack is not an issue, see [2, 3].

3 Efficient Formulas

In the ring of Eisenstein integers, the number 3 is of special interest. It is associated with the square of the prime $\tau = 1 - \omega$, i.e., $3 = -\omega^2\tau^2$. As mentioned earlier, for scalar multiplication $\omega P = \Phi(P)$ of a point P by ω is meaningful which is identified as λP when $P \in G$. In this section, we first derive two efficient formulas for scalar multiplications by τ and 3. Small rational primes that are factorizable also include 7 and 13, i.e., $7 = (2 - \omega)(3 + \omega)$ and $13 = (3 - \omega)(4 + \omega)$. We shall also develop efficient formulas for endomorphisms $2 - \omega, 7$ and $3 - \omega$.

To be more specific, we have

Proposition 3.1. *Let $P = (x, y) \in E_b(\mathbb{F}_p)$.*

1. *The affine coordinates of τP are*
$$\begin{cases} x' = \frac{x^3+4b}{(1-\beta)^2x^2} \\ y' = y\frac{x^3-8b}{(1-\beta)^3x^3}. \end{cases}$$
2. *In Jacobian projective coordinates, τP can be computed using $3\mathbf{S} + 3\mathbf{M}$;*
3. *In Jacobian projective coordinates, $3P$ can be computed using $4\mathbf{S} + 6\mathbf{M}$.*

Proof. 1. For $P = (x, y)$, we have $\omega P = (\beta x, y)$. To compute $\tau P = P - \omega P = (x', y')$, we first see that the slope is
$$\ell = \frac{2y}{(1-\beta)x}.$$
Now notice that $(1 - \beta)^2 = -3\beta$, we have
$$\begin{aligned} x' &= \ell^2 - (1+\beta)x = -\frac{4y^2}{3\beta x^2} - (1+\beta)x \\ &= -\frac{\big(4 + 3\beta(1+\beta)\big)x^3 + 4b}{3\beta x^2} = \frac{x^3+4b}{(1-\beta)^2x^2}. \end{aligned}$$
$$\begin{aligned} y' &= \ell(x - x') - y = \frac{2y}{(1-\beta)x}\left(\frac{3\beta x^3}{3\beta x^2} + \frac{x^3+4b}{3\beta x^2}\right) - y \\ &= y\frac{2(3\beta+1)x^3 - 3\beta(1-\beta)x^3 + 8b}{3\beta(1-\beta)x^3} \\ &= -y\frac{x^3-8b}{3\beta(1-\beta)x^3} = y\frac{x^3-8b}{(1-\beta)^3x^3}. \end{aligned}$$

2. To derive a formula in Jacobian coordinates, write $P = (\frac{X}{Z^2} : \frac{Y}{Z^3} : 1)$. Then

$$X' = \frac{(\frac{X}{Z^2})^3 + 4b}{(1-\beta)^2(\frac{X}{Z^2})^2} = \frac{X^3 + 4bZ^6}{((1-\beta)XZ)^2},$$

$$Y' = \frac{Y}{Z^3}\frac{(\frac{X}{Z^2})^3 - 8b}{(1-\beta)^3(\frac{X}{Z^2})^3} = \frac{Y(X^3 - 8bZ^6)}{((1-\beta)XZ)^3},$$

So a Jacobian coordinates (X_τ, Y_τ, Z_τ) for $(1-\omega)P$ can be

$$\begin{aligned} X_\tau &= X^3 + 4bZ^6 \\ Y_\tau &= Y(X^3 - 8bZ^6) \\ Z_\tau &= (1-\beta)XZ \end{aligned}$$

Using $bZ^6 = Y^2 - X^3$ and $(1-\beta)^2 = -3\beta$, this is further reduced to

$$\begin{aligned} X_\tau &= 4Y^2 - 3X^3 \\ Y_\tau &= Y(9X^3 - 8Y^2) = Y(-2X_\tau + 3X^3) \\ Z_\tau &= (1-\beta)XZ = Z\frac{(1-\beta+X)^2 - X^2 + 3\beta}{2} \end{aligned} \tag{12}$$

Computing X_τ requires $2\mathbf{S}+1\mathbf{M}$, computing Y_τ needs $1\mathbf{M}$, and Z_τ needs $1\mathbf{S}+1\mathbf{M}$ with the square computation of $(1-\beta+X)$.

3. By the fact that $3 = (1-\omega)(1-\omega^2)$, we have $3P = (1-\omega^2)(\tau P)$.

   Let $(X_3 : Y_3 : Z_3)$ be the Jacobian projective coordinates of $3P$ computed from $(1-\omega^2)(\tau P)$, similar to the above, we have

$$\begin{aligned} X_3 &= 4Y_\tau^2 - 3X_\tau^3 \\ Y_3 &= Y_\tau(9X_\tau^3 - 8Y_\tau^2) = Y(-2X_3 + 3X_\tau^3) \\ Z_3 &= (1-\beta^2)X_\tau Z_\tau \end{aligned}$$

   We assume the results of X_τ, Y_τ. Then the computation of X_3, Y_3, as in (12), requires $2\mathbf{S} + 2\mathbf{M}$. We can do better for Z_3 here. Notice that

$$Z_3 = (1-\beta^2)X_\tau Z_\tau = (1-\beta^2)X_\tau(1-\beta)XZ = 3X_\tau(XZ),$$

   so without actually computing Z_τ, the calculation of Z_3 costs $2\mathbf{M}$.

   Plus the computation of X_τ, Y_τ, we need $4\mathbf{S} + 6\mathbf{M}$ to get $3P$ from P.

□

Remarks

1. In Jacobian projective coordinates for a general Weierstrass form, a previously known cost for calculating $3P$ for general curves over prime is given in [20, 3]. When it is restricted to the case of $E_b/\mathbb{F}_p$, the cost is $10\mathbf{S} + 5\mathbf{M}$. Our specific formula for curves $E_b/\mathbb{F}_p$ has achieved a lower cost of $4\mathbf{S} + 6\mathbf{M}$.

2. It is also interesting to note that the computation of $2P = (X_2 : Y_2 : Z_2)$ is very close to that of τP in (12), in fact, one has

$$\begin{aligned} X_2 &= X(9X^3 - 8Y^2) \\ Y_2 &= 9X^3(4Y^2 - 3X^3) - 8Y^4 \\ Z_2 &= 2YZ. \end{aligned} \tag{13}$$

   It is noted that the optimal cost for $2P$ is $4\mathbf{S} + 3\mathbf{M}$ [14].

3. Several facts about τ suggest a possibility of developing window τNAF method, which has been very successful for the family of Koblitz curves over binary fields, for the scalar multiplication of curves $E_b/\mathbb{F}_p$. These facts include (1) $N(\tau) > 1$; (2) τP is faster than point doubling; and (3) any integer $k < n$ has an efficient reduction to a compact form of $k_1 + k_2\tau$.

   In the last step of computing the Jacobian coordinates of τP, we need to get

$$Z_\tau = (1 - \beta)XZ.$$

   If ωP is also available, then we can take βX as an extra input to compute $(1 - \beta)X = X - \beta X$. This will save $1\mathbf{M}$ and we call this an *augmented-τ* operation whose cost becomes $2\mathbf{S} + 3\mathbf{M}$.

   We shall propose a window τNAF method for the scalar multiplication of curves $E_b/\mathbb{F}_p$ in the next section. It can be seen that due to some nice properties of the Eisenstein integers, one is able to construct a very efficient pre-computation. The window τNAF can be further optimized by using tripling and augmented-τ operation.

Now we consider the endomorphism $\rho = 2 - \omega$.

Proposition 3.2. *Let $P = (x, y) \in E_b(\mathbb{F}_p)$.*

1. *The affine coordinates of ρP are*

$$\begin{cases} x' = x\left(\frac{-27x^6 - 12(1+3\beta)(2+\beta)x^3y^2 + 16(1+3\beta)y^4}{\beta^4[4y^2 - 3(1-\beta)x^3]^2}\right) \\ y' = y\left(\frac{-81(1+3\beta)(2+\beta)x^9 + 2^2 3^3 7\beta x^6 y^2 - 144(1+3\beta)x^3y^4 + 64y^6}{\beta^6[4y^2 - 3(1-\beta)x^3]^3}\right); \end{cases}$$

2. *In Jacobian projective coordinates, ρP can be computed using $6\mathbf{S}+7\mathbf{M}$;*

3. *In Jacobian projective coordinates, $7P$ can be computed using $12\mathbf{S}+14\mathbf{M}$.*

Proof. 1. We use the formula from Proposition 3.1 in the calculation

$$\rho P = P + \tau P = (x,y) + \left(\frac{x^3+4b}{(1-\beta)^2x^2}, y\frac{x^3-8b}{(1-\beta)^3x^3} \right).$$

With the slope $\kappa = \frac{y\left(1-\frac{x^3-8b}{(1-\beta)^3x^3}\right)}{x-\frac{x^3+4b}{(1-\beta)^2x^2}} = \frac{2y}{(1-\beta)x}\frac{(2+3\beta)x^3-4b}{(1+3\beta)x^3+4b}$, we get

$$\begin{aligned} x' &= \kappa^2 - x - \frac{x^3+4b}{(1-\beta)^2x^2} = \kappa^2 - \frac{(1-3\beta)x^3+4b}{((1-\beta)x)^2} \\ &= \frac{4y^2[(2+3\beta)x^3-4b]^2 - ((1-3\beta)x^3+4b)[(1+3\beta)x^3+4b]^2}{((1-\beta)x[(1+3\beta)x^3+4b])^2} \\ &= x\left(\frac{-27x^6 - 12(1+3\beta)(2+\beta)x^3y^2 + 16(1+3\beta)y^4}{\beta[4y^2-3(1-\beta)x^3]^2} \right). \end{aligned}$$

The y-coordinate of ρP can be directly computed as [2]

$$\begin{aligned} y' &= \kappa(x-\overline{x}) - y = \frac{2y}{(1-\beta)x}\frac{(2+3\beta)x^3-4b}{(1+3\beta)x^3+4b}(x-\overline{x}) - y \\ &= y\left(\frac{-81(1+3\beta)(2+\beta)x^9 + 756\beta x^6y^2 - 144(1+3\beta)x^3y^4 + 64y^6}{[4y^2-3(1-\beta)x^3]^3} \right). \end{aligned}$$

2. To derive a formula in Jacobian coordinates, write $P = (\frac{X}{Z^2} : \frac{Y}{Z^3} : 1)$. Then

$$x' = \frac{X\big(-27X^6 - 12(1+3\beta)(2+\beta)X^3Y^2 + 16(1+3\beta)Y^4\big)}{\beta^4Z^2\big(4Y^2-3(1-\beta)X^3\big)^2},$$

$$y' = \frac{Y\big((81-324\beta)X^9 + 756\beta X^6Y^2 - 144(1+3\beta)X^3Y^4 + 64Y^6\big)}{\beta^6Z^3\big(4Y^2-3(1-\beta)X^3\big)^3}.$$

So a Jacobian coordinates (X_ρ, Y_ρ, Z_ρ) for ρP becomes

$$\begin{aligned} X_\rho &= X\big(-27X^6 - 12(1+3\beta)(2+\beta)X^3Y^2 + 16(1+3\beta)Y^4\big) \\ Y_\rho &= Y\big((81-324\beta)X^9 + 756\beta X^6Y^2 - 144(1+3\beta)X^3Y^4 + 64Y^6\big) \\ Z_\rho &= \beta^2Z\big(4Y^2-3(1-\beta)X^3\big) \end{aligned} \tag{14}$$

[2] Another way of computing the y-coordinate of ρP is to compute $\frac{dx'}{dx}$. In general, let $(k+\ell\omega)P = (R(x), yS(x))$, then $S(x) = \frac{1}{k+\ell\beta}\frac{dR(x)}{dx}$.

We first compute $A = 3X^3, B = 4Y^2, A^2, B^2, C = \beta A = \frac{(A+\beta)^2 - A^2 - \beta^2}{2}, D = \beta B = \frac{(B+\beta)^2 - B^2 - \beta^2}{2}$ with $\mathbf{M} + 6\mathbf{S}$. Then

$$\begin{aligned} X_\rho &= X\big(-3A^2 - (B+3D)(2A - B + C)\big) \\ Y_\rho &= Y\big(3A^2(A - 4C + 7D) + B^2(B - 3A - 9C)\big) \\ Z_\rho &= Z\big(2A - B + C - D\big). \end{aligned}$$

With this arrangement, computing X_ρ, Y_ρ and Z_ρ use $2\mathbf{M}, 3\mathbf{M}$, and $\mathbf{M}$ respectively. So the entire operation of ρP requires $6\mathbf{S} + 7\mathbf{M}$.

3. By the fact that $7 = (2 - \omega)(2 - \omega^2)$, we have $7P = (2 - \omega^2)(\rho P)$.

   Let $(X_7 : Y_7 : Z_7)$ be the Jacobian projective coordinates of $7P$ computed from $(2 - \omega^2)(\rho P)$. In this case, we just need to use β^2 in the position of β in (14). Thus

$$\begin{aligned} X_7 &= X_\rho\big(-27X_\rho^6 - 12(1+3\beta^2)(2+\beta)X_\rho^3 Y_\rho^2 + 16(1+3\beta^2)Y_\rho^4\big) \\ Y_7 &= Y_\rho\big((81 - 324\beta^2)X_\rho^9 + 756\beta^2 X_\rho^6 Y_\rho^2 - 144(1+3\beta^2)X_\rho^3 Y_\rho^4 + 64Y_\rho^6\big) \\ Z_7 &= \beta Z_\rho\big(4Y_\rho^2 - 3(1-\beta^2)X_\rho^3\big) \end{aligned}$$

   Given ρP, the computation of $7P$ also takes $6\mathbf{S} + 7\mathbf{M}$. This means that the computation of $7P$ from P requires $12\mathbf{S} + 14\mathbf{M}$.

□

Remarks

1. We first remark that ρP is less expensive than a point addition as the later costs $11\mathbf{M} + 5\mathbf{S}$. We shall use it in the pre-computation later in the window τNAF. It is also remarked that the $7P$ formula in proposition 3.2 is also efficient. As an independent calculation of $7P$ could be $7P = 2^3P - P$ and it costs $3(5\mathbf{S} + 2\mathbf{M}) + (5\mathbf{S} + 11\mathbf{M}) = 20\mathbf{S} + 17\mathbf{M}$.

2. We note that in finding ρP, one needs to calculate X^3, Y^2 and Y^4 as intermediate result. These terms are also appear in the calculation of $2P$ by using (13). If both ρP and $2P$ are required, they can be proceeded together to save $1\mathbf{M} + 3\mathbf{S}$.

3. We also remark that in forming the affine coordinates of ρP (similar to τP), we manage to make the denominators of x and y in the way so that the conversion to Jacobian projective coordinates becomes more natural.

For the endomorphism $\sigma = 3 - \omega$, we just state the calculation result of σP here:

Proposition 3.3. *Let $P = (x, y) \in E_b(\mathbb{F}_p)$. The affine coordinates of $\sigma P = (3-\omega)P$ are*

$$\begin{aligned} x' &= \frac{\beta x^{16} + (-336 - 199\beta)bx^{13} + (-480 - 1400\beta)b^2x^{10} + (-720 - 2672\beta)b^3x^7 + (-1344 - 1216\beta)b^4x^4}{y^2[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^2} \\ &\quad + \frac{(-768 + 256\beta)b^5x}{y^2[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^2} \\ y' &= \frac{-x^{24} + (298 + 756\beta)bx^{21} + (-18649 - 1944\beta)b^2x^{18} + (-65396 - 26892\beta)b^3x^{15} + (-36016 - 38016\beta)b^4x^{12}}{y^3[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^3} \\ &\quad + \frac{(69568 - 62208\beta)b^5x^9 + (76544 - 103680\beta)b^6x^6 + (13312 - 55296\beta)b^7x^3 - 4096b^8}{y^3[(4+3\beta)x^6 + 4(5-6\beta)bx^3 + 16b^2]^3} \end{aligned}$$

At the end of this section, we include a table to summarize the costs of some efficient endomorphisms we have discussed:

Table 2: Costs of Endomorphisms

Endomorphism	Our Formula in This Section	Current Known Result	Remark
$\tau = 1 - \omega$	$3\mathbf{S} + 3\mathbf{M}$	–	Better than a point doubling
3	$4\mathbf{S} + 6\mathbf{M}$	$10\mathbf{S} + 5\mathbf{M}$	
$\rho = 2 - \omega$	$6\mathbf{S} + 7\mathbf{M}$	–	Better than a point addition
7	$12\mathbf{S} + 14\mathbf{M}$	$20\mathbf{S} + 17\mathbf{M}$	

4 Window τNAF Method

We have found an efficient endomorphism $\tau = 1 - \omega$ for E_b whose norm is $N(\tau) = 3 > 1$. It is a desired candidate to create an efficient window τNAF method as it satisfies the basic requirements posed in [13].

In [18], Koblitz considered the following supersingular elliptic curve over finite fields of characteristic 3,

$$E_{3,a} : y^2 = x^3 - x - (-1)^a / \mathbb{F}_{3^m},\ a \in \{0, 1\}. \tag{15}$$

The Frobenius map for this case is given as $\tau_3(x, y) = (x^3, y^3)$ (for the point at infinity, $\tau_3(\mathcal{O})$ is set to be $\mathcal{O}$). The characteristic polynomial of τ_3 is $T^2 - 3(-1)^a T + 3$, so $\mathbb{Z}[\omega]$ is also the ring of endomorphisms of $E_{3,a}$. Even though τ_3 and τ are different endomorphisms for different elliptic curves, they are the same as complex

numbers. Koblitz proves the existence and uniqueness of a nonadjacent form of τ_3-adic expansion for an integer in $\mathbb{Z}[\tau_3] = \mathbb{Z}[\omega]$ and uses the form to perform fast scalar multiplication in $E_{3,a}(\mathbb{F}_{3^m})$. The scalar multiplication using window τ_3NAF method for $E_{3,a}$ is proposed by Blake, Murty and Xu, where a reduction is described and a termination proof of the window τ_3NAF algorithm is obtained [4]. Some ideas of [4] are further developed in this section, but due to the different nature of $E_b/\mathbb{F}_p$ and $E_{3,a}/\mathbb{F}_{3^m}$, we use the reduction illustrated in section 2. For the case of window size being 4, an efficient pre-computation of τNAF for $E_b(\mathbb{F}_p)$ is carefully designed. Note that there is no concrete pre-computation discussed in [4]. In this paper, the efficient tripling ($3P$) formula and augmented-τ are used to achieve further gain.

Since $\tau = 1 - \omega$, the ring of Eisenstein ring $\mathbb{Z}[\omega]$ can be also written as

$$\mathbb{Z}[\tau] = \{x + y\tau : x, y \in \mathbb{Z}\}.$$

Given a natural number w, the idea of the width w window τNAF method is to seek the following expansion (*window τNAF*) of an element $k_1 + k_2\tau \in \mathbb{Z}[\omega]$:

$$k_1 + k_2\tau = \sum_{j=0}^{N} c_j \tau^j \tag{16}$$

with the property that each nonzero c_j is taken from a suitable set C called the *set of coefficients* and each segment $\{c_j, c_{j+1}, \ldots, c_{j+w-1}\}$ contains at most one nonzero element. To compute $(k_1 + k_2\tau)P$, one first sets up a pre-computation: perform cP for each $c \in C$, and then evaluates $\sum_{j=0}^{N} \tau^j(c_j P)$.

4.1 Criterion of Divisibility by τ^w

To derive (16), a criterion of the divisibility of elements in $\mathbb{Z}[\tau]$ by τ^w is useful. The next lemma is from [4, 6] where a proof was outlined. Since it is crucial in our discussion, we provide a more detailed proof here.

Lemma 4.1. *Let w be a positive integer and $\delta_w = \lceil \frac{w}{2} \rceil - \lfloor \frac{w}{2} \rfloor$, then*

1. $$\tau^w = (-3\omega)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w}$$

2. *For $x + y\tau \in \mathbb{Z}[\tau]$,*

$$\tau^w | x + y\tau \iff 3^{\lceil \frac{w}{2} \rceil} | x \text{ and } 3^{\lfloor \frac{w}{2} \rfloor} | y.$$

Proof. 1. Since $\tau^2 = -3\omega$, we have

$$\tau^w = (\tau^2)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w} = (-3\omega)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w}.$$

2. If $\tau^w | x + y\tau$, then $3^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w} | x + y\tau$, because $(-\omega)^{\lfloor \frac{w}{2} \rfloor}$ is a unit. Thus, there exists $\alpha + \beta\tau \in \mathbb{Z}[\tau]$, such that

$$x + y\tau = 3^{\lfloor \frac{w}{2} \rfloor}(\alpha\tau^{\delta_w} + \beta\tau^{1+\delta_w}).$$

As $\delta_w = \begin{cases} 0 & \text{if } w \text{ is even} \\ 1 & \text{if } w \text{ is odd} \end{cases}$, we see that

$$x = 3^{\lfloor \frac{w}{2} \rfloor}\alpha = 3^{\lceil \frac{w}{2} \rceil}\alpha, \quad y = 3^{\lfloor \frac{w}{2} \rfloor}\beta$$

when w is even. If w is odd, since $\tau^2 = -3 + 3\tau$, we get

$$x = -3^{\lfloor \frac{w}{2} \rfloor + 1}\beta = -3^{\lceil \frac{w}{2} \rceil}\beta, \quad y = 3^{\lfloor \frac{w}{2} \rfloor}(\alpha + 3\beta).$$

Therefore, in either case

$$3^{\lceil \frac{w}{2} \rceil} | x \text{ and } 3^{\lfloor \frac{w}{2} \rfloor} | y.$$

Conversely, if $x = 3^{\lceil \frac{w}{2} \rceil} f, y = 3^{\lfloor \frac{w}{2} \rfloor} g$ in $\mathbb{Z}$, then

$$\begin{aligned} x + y\tau &= 3^{\lfloor \frac{w}{2} \rfloor}(3^{\delta_w} f + g\tau) = 3^{\lfloor \frac{w}{2} \rfloor}(-\omega)^{\lfloor \frac{w}{2} \rfloor} \tau^{\delta_w} \frac{3^{\delta_w} f + g\tau}{\tau^{\delta_w}} (-\omega)^{-\lfloor \frac{w}{2} \rfloor} \\ &= \tau^w \frac{3^{\delta_w} f + g\tau}{\tau^{\delta_w}} (-\omega)^{-\lfloor \frac{w}{2} \rfloor}. \end{aligned}$$

The result is proved as $\frac{3^{\delta_w} f + g\tau}{\tau^{\delta_w}} (-\omega)^{-\lfloor \frac{w}{2} \rfloor} \in \mathbb{Z}[\tau]$ due to the fact that $\tau | 3$.

□

The following corollary is useful in designing an efficient set of coefficients.

Corollary 4.1. *Let $w > 1$ and $a + b\tau \not\equiv 0 \pmod{\tau}$, then for any $u \in U \setminus \{1\}$,*

$$a + b\tau \not\equiv u(a + b\tau) \pmod{\tau^w}.$$

Proof. Write $x + y\tau = a + b\tau - u(a + b\tau)$, then a direct calculation gives

$$x + y\tau = \begin{cases} 2a + 2b\tau & \text{if } u = -1; \\ -3b + (3b + a)\tau & \text{if } u = \omega; \\ (2a + 3b) - (a + b)\tau & \text{if } u = -\omega; \\ 3(a + b) - a\tau & \text{if } u = \omega^2; \\ -(a + 3b) + (a + 2b)\tau & \text{if } u = -\omega^2. \end{cases}$$

Since $3 \not| a$, it is easy to see that $3^{\lceil \frac{w}{2} \rceil} \not| x$ or $3^{\lfloor \frac{w}{2} \rfloor} \not| y$. □

4.2 Pre-computation

We need to decide a set for the coefficients of the representation (16).

Consider the set of all elements of $\mathbb{Z}[\tau]$ which are not divisible by τ. By lemma 4.1, a set of representatives of congruence classes of such elements modulo τ^w is

$$R = \{x + y\tau : 0 \leq x \leq 3^{\lceil \frac{w}{2} \rceil} - 1, 0 \leq y \leq 3^{\lfloor \frac{w}{2} \rfloor} - 1, \text{ and } 3 \nmid x\}.$$

For each $x + y\tau \in R$, let

$$C_{x,y} = \{g + h\tau \in \mathbb{Z}[\tau] : g \equiv x \pmod{3^{\lceil \frac{w}{2} \rceil}},\ h \equiv y \pmod{3^{\lfloor \frac{w}{2} \rfloor}}, N(g + h\tau) < 3^w\}.$$

Since $\mathbb{Z}[\tau]$ is a Euclidean ring, $C_{x,y}$ is nonempty. In fact, $C_{x,y}$ usually contains more than one element. This is a useful property as one has a flexibility in selecting an element in $C_{x,y}$ that contributes to a better pre-computation to serve as a coefficient of the expansion (16).

We choose one element $\tilde{x} + \tilde{y}\tau$ from each $C_{x,y}$, then a set (nonzero coefficients for the expression (16)) can be formed as follows:

$$C = \{\tilde{x} + \tilde{y}\tau : 0 \leq x \leq 3^{\lceil \frac{w}{2} \rceil} - 1, 0 \leq y \leq 3^{\lfloor \frac{w}{2} \rfloor} - 1, \text{ and } 3 \nmid x\}. \tag{17}$$

We can use Algorithm 3.1 of [4] to produce (16). The correctness of this algorithm is examined in [4] (Theorem3.1): because $\mathbb{Z}[\omega]$ is Euclidean and imaginary quadratic, the width w window τNAF method terminates. We include this algorithm below.

Algorithm 1 Width w window τ-NAF Method

Require: an element $\eta = a + b\tau$ of $\mathbb{Z}[\tau]$
Ensure: S, the array of coefficients of window τ-NAF of η

```
1:  function GEN-NAF(a, b)
2:      S ← <>
3:      while a ≠ 0 or b ≠ 0 do
4:          if 3 ∤ a then
5:              x ← a (mod 3^⌈w/2⌉)
6:              y ← b (mod 3^⌊w/2⌋)
7:              a ← a − x̃
8:              b ← b − ỹ
9:              prepend x̃ + ỹτ to S
10:         else
11:             prepend 0 to S
12:         end if
```

$t \leftarrow a$
$a \leftarrow a + b$
$b \leftarrow \frac{-t}{3}$
end while
return S
end function

4.2.1 A pre-computation for $w = 4$

We now describe an explicit efficient pre-computation for $w = 4$.

Let $w = 4$. In this case, we need to find easily computable coefficients $\tilde{x} + \tilde{y}\tau$ from each $C_{x,y}$ for $x = 1, 2, 4, 5, 7, 8$ and $y \in \{0, 1, 2, \cdots, 8\}$. It is noted that only $x = 1, 2, 4$ should be considered, as

$$-(x + y\tau) \equiv (9 - x) + (9 - y)\tau \pmod{\tau^4}.$$

The part for $x = 5, 7, 8$ is obtained by taking negation. This means that we only need to determine 27 coefficients and a half of the pre-computations is saved. As can be seen later, there are actually 9 pre-computations that need to be processed, the rest 18 can be obtained easily. More specifically, with each $x + y\tau$ ($x = 1, 2, 4$ and $0 \leq y < 9$) serving as an index for the corresponding coefficient $\tilde{x} + \tilde{y}\tau$, we first fix 9 coefficients

$$S = \{1, 2, 4, 1 + \tau, 2 + 2\tau, 1 + 2\tau, 2 + 4\tau, 2 + \tau, 1 - 2\tau\},$$

these coefficients are indexed by $1, 2, 4, 1 + \tau, 2 + 2\tau, 1 + 2\tau, 2 + 4\tau, 2 + \tau, 1 + 7\tau$. The set of (nonzero) coefficients for (16) is given by

$$C = S \cup (-S) \cup \omega S \cup (-\omega S) \cup \omega^2 S \cup (-\omega^2 S).$$

By corollary 4.1, the set C represents all classes (modulo τ^4) of elements that are not divisible by τ. Obviously C is U-invariant.

In table 3, the first, third and fifth row are all indexes $x + y\tau$'s for $x = 1, 2, 4$ and $0 \leq y < 9$. The second row lists the set S, with their corresponding indexes $x + y\tau$ in the first row. An element $\tilde{x} + \tilde{y}\tau$ in the forth row is obtained by multiplying ω to the corresponding (column) element in the second row if $\tilde{x} + \tilde{y}\tau \in C_{x,y}$ with $1 \leq x \leq 4$, otherwise, $\tilde{x} + \tilde{y}\tau$ is obtained by multiplying $-\omega$ to the corresponding (column) element in the second row. The corresponding representative $x + y\tau$ is put in the third row as its index. We form the sixth row and fifth row in a similar way by multiplying ω^2 or $-\omega^2$ to the corresponding (column) element in the second row.

Table 3: Pre-Computation for $w = 4$

$x + y\tau$	1	2	4	$1 + \tau$	$2 + 2\tau$	$1 + 2\tau$	$2 + 4\tau$	$2 + \tau$	$1 + 7\tau$
$\tilde{x} + \tilde{y}\tau$	1	2	4	$1 + \tau$	$2 + 2\tau$	$1 + 2\tau$	$2 + 4\tau$	$2 + \tau$	$1 - 2\tau$
$x + y\tau$	$1 + 8\tau$	$2 + 7\tau$	$4 + 5\tau$	$4 + 6\tau$	$1 + 6\tau$	$2 + 5\tau$	$4 + \tau$	$4 + 4\tau$	$4 + 3\tau$
$\tilde{x} + \tilde{y}\tau$	$1 - \tau$	$2(1 - \tau)$	$4(1 - \tau)$	$4 - 3\tau$	$-8 + 6\tau$	$-7 + 5\tau$	$-14 + 10\tau$	$-5 + 4\tau$	$-5 + 3\tau$
$x + y\tau$	$2 + 8\tau$	$4 + 7\tau$	$1 + 4\tau$	$4 + 2\tau$	$1 + 5\tau$	$1 + 3\tau$	$2 + 6\tau$	$2 + 3\tau$	$4 + 8\tau$
$\tilde{x} + \tilde{y}\tau$	$2 - \tau$	$4 - 2\tau$	$-8 + 4\tau$	$-5 + 2\tau$	$10 - 4\tau$	$-8 + 3\tau$	$-16 + 6\tau$	$-7 + 3\tau$	$4 - \tau$

Precisely, coefficients $\tilde{x}+\tilde{y}\tau$ in the fourth row in table 3 are obtained by multiplying ω or $-\omega$ to the ones in the second row in the following manner:

$$\begin{array}{lll} 1 - \tau = \omega & 2(1 - \tau) = 2\omega, & 4(1 - \tau) = 4\omega \\ 4 - 3\tau = \omega(1 + \tau), & -8 + 6\tau = -\omega(2 + 2\tau), & -7 + 5\tau = -\omega(1 + 2\tau) \\ -14 + 10\tau = -\omega(2 + 4\tau), & -5 + 4\tau = -\omega(2 + \tau) & -5 + 3\tau = \omega(1 - 2\tau). \end{array}$$

Similarly, $\tilde{x} + \tilde{y}\tau$'s in the sixth row in table 3 are obtained :

$$\begin{array}{lll} 2 - \tau = -\omega^2 & 4 - 2\tau = 2(-\omega^2), & -8 + 4\tau = 4\omega^2 \\ -5 + 2\tau = \omega^2(1 + \tau), & 10 - 4\tau = -\omega^2(2 + 2\tau), & -8 + 3\tau = \omega^2(1 + 2\tau) \\ -16 + 6\tau = \omega^2(2 + 4\tau), & -7 + 3\tau = \omega^2(2 + \tau), & 4 - \tau = \omega^2(1 - 2\tau). \end{array}$$

We denote C^+ the set of coefficients appeared in table 3 (the second, forth and sixth rows). Then

$$C = C^+ \cup (-C^+).$$

Given a point P, the pre-computation is to compute $Q_{x+y\tau} = (\tilde{x} + \tilde{y}\tau)P$ for $x + y\tau \in C^+$ (i.e. $x = 1, 2, 4$ and $0 \le y < 9$).

From the construction, we see that if a point $Q = (\tilde{x} + \tilde{y}\tau)P$ for the second row is computed, then the corresponding pre-computation in the fourth row and sixth row can be simply obtained as ωQ (or $-\omega Q$) and $\omega^2 Q$ (or $-\omega^2 Q$) , respectively. Note that computing ωQ is easy: if $Q = (r, s)$, then $\omega Q = (\beta r, s)$; computing $\omega^2 Q$ can even be neglected in the sense that

$$\omega^2 Q = (\beta^2 r, s) = (-\beta r - r, s),$$

since the multiplication βr has been computed in ωQ.

Concretely, we perform $Q_{x+y\tau} = (\tilde{x} + \tilde{y}\tau)P$ for the second row one by one in the following order:

$$\begin{array}{lll} Q_2 = 2P, & Q_4 = 4P = 2Q_2, & Q_{1+\tau} = (1 + \tau)P = \rho P \\ Q_{2+2\tau} = (2 + 2\tau)P = 2Q_{1+\tau}, & Q_{2+\tau} = (2 + \tau)P = Q_{1+\tau} + P, & Q_{1+2\tau} = (1 + 2\tau)P = Q_{2+2\tau} - P \\ Q_{1+7\tau} = (1 - 2\tau)P = Q_2 - Q_{1+2\tau} & Q_{2+4\tau} = (2 + 4\tau)P = 2Q_{1+2\tau} & \end{array}$$

In this process, we compute $2P$ and ρP together to use $16\mathbf{M}$ (as they share the computations of X^3, Y^2, Y^4). The rest requires $3\mathbf{DBL} + 3\mathbf{ADD}$. The computation for the fourth row needs 9 field multiplications. So the pre-computation cost is

$$16\mathbf{M} + 9\mathbf{M} + 3 \cdot 7\mathbf{M} + 3 \cdot 16\mathbf{M} = 94\mathbf{M}.$$

4.3 Width-4 Window τNAF Method for Scalar Multiplication

In coordination with the pre-computation designed above, we choose window size $w = 4$ for scalar multiplication. This is an appropriate size one because it also helps to maximize the benefit brought by the more efficient $3P$ formula. We have determined the set C^+ of coefficients in section 4.2.1 for pre-computation. The pre-computation with respect to a point $P \in E_b(\mathbb{F}_p)$ is to compute cP for every $c \in C^+$. The scalar multiplication with negative part of C^+ is trivial.

Now we describe our scalar multiplication algorithm for curves $E_b/\mathbb{F}_p$ over prime fields. Given a scalar $k < n$ and a point P of order n, we first use (10) to get reduction $k = k'_1 + k'_2\lambda \pmod{n}$. This implies that $kP = (k'_1 + k'_2\omega)P$. Let $k_1 = k'_1 + k'_2, k_2 = -k'_2$, then

$$kP = (k_1 + k_2\tau)P.$$

Next we use Algorithm 1 to compute

$$k_1 + k_2\tau = \sum_{i=0}^{\ell-1} \varepsilon_i c_i \tau^i \tag{18}$$

where $c_i \in C^+ \cup \{0\}, \varepsilon_i \in \{-1, 1\}$. The window τNAF algorithm is

Algorithm 2 Window τ-NAF Scalar Multiplication Method for $E_b/\mathbb{F}_p$

Require: a positive integer $k < n$ and a point $P \in E_b(\mathbb{F}_p)$ of order n
Ensure: the scalar multiplication kP

```
1:  function SCAL-MUL(k, P)
2:      Pre-Computation: cP for each c ∈ C^+.
3:      Get expression (18) for k
4:      R ← O
5:      for i from ℓ − 1 downto 0 do
6:          R ← τR
7:          if c_i ≠ 0 then
8:              if ε_i = 1 then
9:                  R ← R + c_i P
10:             else
11:                 R ← R − c_i P
12:             end if
13:         end if
14:     end for
```

15: return R
16: **end function**

4.3.1 Optimization

The expression (18) can be written as

$$k_1 + k_2\tau = \varepsilon_{\ell_0} c_{\ell_0} \tau^{\ell_0} + \varepsilon_{\ell_1} c_{\ell_1} \tau^{\ell_1} + \cdots + \varepsilon_{\ell_{t-1}} c_{\ell_{t-1}} \tau^{\ell_{t-1}}$$

where the nonzero coefficient $c_{\ell_j} \in C^+$, $\varepsilon_{\ell_j} \in \{1, -1\}$, and $\ell_j - \ell_{j-1} \geq 4$.

By lemma 4.1, $\tau^\ell = (-3\omega)^{\lfloor \frac{\ell}{2} \rfloor} \tau^{\delta_\ell}$ where $\delta_\ell = \lceil \frac{\ell}{2} \rceil - \lfloor \frac{\ell}{2} \rfloor$, the above expression of $k_1 + k_2\tau$ can rewritten as

$$\begin{aligned} k_1 + k_2\tau &= \varepsilon_{\ell_0} c_{\ell_0} (-\omega)^{\lfloor \frac{\ell_0}{2} \rfloor} \tau^{\delta_{\ell_0}} 3^{\lfloor \frac{\ell_0}{2} \rfloor} + \varepsilon_{\ell_1} c_{\ell_1} (-\omega)^{\lfloor \frac{\ell_1}{2} \rfloor} \tau^{\delta_{\ell_1}} 3^{\lfloor \frac{\ell_1}{2} \rfloor} + \\ &\quad + \cdots + \varepsilon_{\ell_{t-1}} c_{\ell_{t-1}} (-\omega)^{\lfloor \frac{\ell_{t-1}}{2} \rfloor} \tau^{\delta_{\ell_{t-1}}} 3^{\lfloor \frac{\ell_{t-1}}{2} \rfloor} \\ &= \varepsilon'_{\ell_0} c'_{\ell_0} \tau^{\delta_{\ell_0}} 3^{\lfloor \frac{\ell_0}{2} \rfloor} + \varepsilon'_{\ell_1} c'_{\ell_1} \tau^{\delta_{\ell_1}} 3^{\lfloor \frac{\ell_1}{2} \rfloor} + \cdots + \varepsilon'_{\ell_{t-1}} c'_{\ell_{t-1}} \tau^{\delta_{\ell_{t-1}}} 3^{\lfloor \frac{\ell_{t-1}}{2} \rfloor} \end{aligned} \quad (19)$$

Here $\varepsilon'_{\ell_j} c'_{\ell_j} = \varepsilon_{\ell_j} c_{\ell_j} (-\omega)^{\lfloor \frac{\ell_j}{2} \rfloor} \in C$ since C is U-invariant, and the element $\varepsilon'_{\ell_j} \in \{1, -1\}$ is chosen so that $c'_{\ell_j} \in C^+$. The new expression (19) makes the use of the efficient tripling possible (recall that $3R$ can be done by using only $10\mathbf{M}$ for any point R). In the calculation of kP, the term $\varepsilon'_{\ell_j} c'_{\ell_j} \tau^{\delta_{\ell_j}} P$ is processed as $\tau^{\delta_{\ell_j}}(\varepsilon'_{\ell_j} c'_{\ell_j} P) = \tau^{\delta_{\ell_j}}(Q_{x+y\tau})$ where $Q_{x+y\tau} = \varepsilon'_{\ell_j} c'_{\ell_j} P$ has already been include in pre-computation. By the U-invariant property, if $Q_{x+y\tau}$ is pre-computed, so is $\omega Q_{x+y\tau}$. Thus if $\delta_{\ell_j} = 1$, one can use augmented-τ to compute $\tau^{\delta_{\ell_j}}(Q_{x+y\tau})$ using $5\mathbf{M}$, which is half of that for tripling.

The optimized version of our window τNAF algorithm is

Algorithm 3 Optimized Window τNAF Scalar Multiplication Method for $E_b/\mathbb{F}_p$

Require: a positive integer $k < n$ and a point $P \in E_b(\mathbb{F}_p)$ of order n
Ensure: the scalar multiplication kP
1: **function** OPT-SCAL-MUL(k, P)
2: Pre-Computation: cP for each $c \in C^+$.
3: Get expression (19) for k
4: $R \leftarrow \mathcal{O}$
5: **for** i from $\lfloor \frac{\ell_{t-1}}{2} \rfloor$ downto 0 **do**
6: $R \leftarrow 3R$
7: **if** $i = \lfloor \frac{\ell_j}{2} \rfloor$ for some j **then**
8: $Q \leftarrow c'_{\ell_j} P$

```
            if δ_{ℓ_j} = 1 then
                Q ← τQ    (*Use augmented-τ since ωQ is available*)
            end if
            if ε'_{ℓ_j} = 1 then
                R ← R + Q
            else
                R ← R − Q
            end if
        end if
    end for
    return R
end function
```

4.3.2 Estimation of the cost for width-4 window τNAF

In the process of calculating kP for $k < n$, one reduces k to $k_1 + k_2\tau$ such that $k \equiv k_1 + k_2\tau \pmod{n}$ with $|k_1| \lessapprox \frac{4}{3}\sqrt{n}, |k_2| \lessapprox \frac{2}{3}\sqrt{n}$, as in (10).

Note that $N(\tau) = 3$, the window τNAF for $k_1 + k_2\tau$ is expected to be of length $\log_3 n$. In the expansion (16) for $k_1 + k_2\tau$, after each nonzero coefficient, there will be w consecutive zeros, beyond that, asymptotic expectation of zeros followed is $\frac{1}{3} + \frac{1}{3^2} + \cdots = \frac{1}{2}$. So the average number of nonzero terms is

$$\frac{\log_3 n}{w + \frac{1}{2}}.$$

With $w = 4$, this means that, given a pre-computation, algorithm 2 requires $2\frac{\log_3 n}{4.5}$ **tripling**, $(\log_3 n - 4\frac{\log_3 n}{4.5})$ **augmented-τ**, and $\frac{\log_3 n}{4.5}$**ADD**.

Recall that **augmented-τ** = 5**M**, **tripling** = 10**M**, and **ADD** = 16**M**, so the overall cost for the computation of kP in terms of the number of field multiplications becomes

$$\left(\frac{2}{4.5} \cdot 10 \quad + (1 - \tfrac{4}{4.5}) \cdot 5 + \tfrac{1}{4.5} \cdot 16\right) \log_3 n\mathbf{M} + 94\mathbf{M} < (5.40 \log_2 n + 94)\mathbf{M}. \tag{20}$$

Currently, the best method for scalar multiplication for curves $E_b/\mathbb{F}_p$ is the GLV method. According to our experiments, the version of interleaving with 4-NAF performs the best for SECP256K1. For $E_b/\mathbb{F}_p$, the interleaving with w-NAF is to compute $k_1P + k_2Q$ where $Q = \lambda P$ using w-NAF to k_1 and k_2. The pre-computation can be done essentially just for P: pre-compute jP for $j = 3, 5, \cdots, 2^{w-1} - 1$. The

Q part of pre-computation is obtained simply by $jQ = \lambda(jP)$ by using 4 field multiplications for $j = 1, 3, 5, 7$. The rest of the cost of interleaving with 4-NAF is $\frac{\log_2 n}{2}\mathbf{DBL} + \frac{\log_2 n}{5}\mathbf{ADD}$. Converting to field multiplications, the cost becomes

$$\left(\frac{\log_2 n}{2} + 1\right) \cdot 7\mathbf{M} + \left(\frac{\log_2 n}{5} + 3\right) \cdot 16\mathbf{M} + 4\mathbf{M} \approx (6.7 \log_2 n + 59)\mathbf{M}. \tag{21}$$

Comparing (20) with (21), we see that the width-4 window τ-NAF method achieves more than $16.7\%, 17.6\%$ and 18% of improvements over the GLV method, when $\log_2 n = 256, 384$ and 512 respectively. Table 4 lists precisely the numbers of $\mathbf{M}$ used for the three sizes, for both of our optimized window *tau*NAF method and the GLV method.

Table 4: Costs of Field Multiplication Used

Bit size of the Order of Group	Our Methdod	GLV Method
256	1476**M**	1774**M**
384	2168**M**	2631**M**
512	2859**M**	3489**M**

4.3.3 Regular window τNAF

In [16], Joye and Tunstall describe regular recoding algorithms for integers to implement side-channel resistant exponentiation algorithms. This idea is extended to the case of τNAF recoding for Koblitz curves by Oliveira, Aranha, López, and Rodríguez-Henríquez[23] (see also [24]).

The regular window τNAF representation of a scalar k is the following sparse τ expansion of its reduction $k_1 + k_2\tau$:

$$k_1 + k_2\tau = \sum_{i=m}^{\ell-1} u_i(\tau^w)^i, \tag{22}$$

where the coefficients $u_i \neq 0$. For Koblitz curves E_a, this is achieved in [23] by taking a set of coefficients given in [28] for τNAF with window size $w + 1$. The essential property of this set of coefficients is that the 2^w coefficients lie in different equivalent classes modulo τ^{w+1}, and they are divided into 2^{w-1} where the two coefficients in each pair are congruent modulo τ^w.

Yu and Xu [32] use another approach by choosing two sets of coefficients for τNAF with window size w, so the Euclidean property of $\mathbb{Q}(\sqrt{-7})$[3] can be used to guarantee the correctness of coefficient selection process [4]. The union of the these two sets of coefficients shares the same essential property as we just described for the construction in [23].

Here we follow the idea in [32] to create two coefficient sets that satisfy certain condition, for the case of window size $w = 4$, to obtain regular window τNAF representation for the curves $E_b/\mathbb{F}_p$. More specific, for each pair of (x, y) with $0 \le x \le 8, 0 \le y \le 8,$ and $3 \nmid x$, we select two elements $U_{x+y\tau} = \tilde{x}+\tilde{y}\tau$, $V_{x+y\tau} = \tilde{\tilde{x}}+\tilde{\tilde{y}}\tau$ from the set

$$C_{x,y} = \{g + h\tau \in \mathbb{Z}[\tau] : g \equiv x \pmod{3^2}, h \equiv y \pmod{3^2}, N(g + h\tau) < 3^4\}.$$

To form a regular expression (22) for $w = 4$, it is required that

$$\tau^5 \nmid (V_{x+y\tau} - U_{x+y\tau}). \tag{23}$$

Note that $\tau^4 | (V_{x+y\tau} - U_{x+y\tau})$, we see that the elements in the set

$$\{V_{x+y\tau}, U_{x+y\tau} : 0 \le x \le 8, 0 \le y \le 8, \text{ and } 3 \nmid x\}$$

are all distinct modulo τ^5, and there are 54 congruent modulo τ^4 pairs of $(V_{x+y\tau}, U_{x+y\tau})$.

By lemma 4.1, the condition (23) is satisfied if $\tilde{x} \not\equiv \tilde{\tilde{x}} \pmod{3^3}$.

For one set, we use the set $C = \{\tilde{x} + \tilde{y}\tau : 0 \le x \le 8, 0 \le y \le 8, \text{ and } 3 \nmid x\}$ from section 4.2. The other set of coefficients is $C' = C^+ \cup (-C'^+)$ where

$$\begin{aligned} C'^+ &= \{\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau : 0 \le x \le 4, 0 \le y \le 8, \text{ and } 3 \nmid x\} \\ &= \{-8, -7, -5, -8+\tau, -7+2\tau, -8+2\tau, -7+4\tau, -7+\tau, -8+7\tau\} \cup \\ &\quad \{-8+8\tau, -7+7\tau, -5+5\tau, -5+6\tau, 1-3\tau, 2-4\tau, -5+\tau, 4-5\tau\} \cup \\ &\quad \{-16+8\tau, -14+7\tau, 10-5\tau, 13-7\tau, -8+5\tau, 10-6\tau, 2-3\tau, 11-6\tau, -5-\tau\}. \end{aligned}$$

The set C' is also U-invariant.

The pre-computation order is arranged in the way that is similar to table 3:

Similarly, we only need to compute all $Q'_{x+y\tau} = (\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau)P$ for row 2 in table 5 as

$Q'_1 = -8P = -2Q_4$	$Q'_2 = -7P = Q'_1 + P$	$Q'_4 = -5P = -Q_4 - P$
$Q'_{1+\tau} = (-8+\tau)P = -Q_{1+8\tau} + Q'_2$	$Q'_{2+2\tau} = (-7+2\tau)P = -Q_{2+7\tau} + Q'_4$	$Q'_{1+2\tau} = (-8+2\tau)P = -2Q_{4+8\tau}$
$Q'_{2+4\tau} = (-7+4\tau)P = Q_{4+4\tau} - Q_2$	$Q'_{2+\tau} = (-7+\tau)P = Q'_{1+\tau} + P$	$Q'_{1+7\tau} = (-8+7\tau)P = Q'_{2+7\tau} - P$

[3]Recall that the Frobenius endomorphism for Koblitz curve $E_a/\mathbb{F}_{2^m}$ is identified as the complex number $\frac{(-1)^{1-a}+\sqrt{-7}}{2}$.

Table 5: Another Pre-Computation for $w = 4$

$x + y\tau$	1	2	4	$1+\tau$	$2+2\tau$	$1+2\tau$	$2+4\tau$	$2+\tau$	$1+7\tau$
$\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau$	-8	-7	-5	$-8+\tau$	$-7+2\tau$	$-8+2\tau$	$-7+4\tau$	$-7+\tau$	$-8+7\tau$
$x + y\tau$	$1+8\tau$	$2+7\tau$	$4+5\tau$	$4+6\tau$	$1+6\tau$	$2+5\tau$	$4+\tau$	$4+4\tau$	$4+3\tau$
$\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau$	$-8+8\tau$	$-7+7\tau$	$-5+5\tau$	$-5+6\tau$	$1-3\tau$	$2-4\tau$	$-5+\tau$	$4-5\tau$	$13-6\tau$
$x + y\tau$	$2+8\tau$	$4+7\tau$	$1+4\tau$	$4+2\tau$	$1+5\tau$	$1+3\tau$	$2+6\tau$	$2+3\tau$	$4+8\tau$
$\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau$	$-16+8\tau$	$-14+7\tau$	$10-5\tau$	$13-7\tau$	$-8+5\tau$	$10-6\tau$	$2-3\tau$	$11-6\tau$	$-5-\tau$

This requires $2\mathbf{DBL} + 7\mathbf{ADD}$. Pre-computation with respect to rows 4 and 6 are obtained by multiplying β (or $-\beta$) and β^2 (or $-\beta^3$) to the x-coordinate of the above $Q'_{x+y\tau}$s. Since $\beta^2 = -\beta - 1$, we need additional $9\mathbf{M}$.

To get a τ expansion that is regular, we need to eliminate 0 from the coefficient set. To this end, we replace lines 7-9 of Algorithm 1 by

---

if $((a = \tilde{x})$ and $(b = \tilde{y}))$ or $((a = \tilde{\tilde{x}})$ and $(b = \tilde{\tilde{y}}))$ **then**
    prepend $a + b\tau$ to S, break
if $3^3 | (a - \tilde{x})$ **then**
    $a \leftarrow a - \tilde{\tilde{x}}$, $b \leftarrow b - \tilde{\tilde{y}}$
    prepend $\tilde{\tilde{x}} + \tilde{\tilde{y}}\tau$ to S
else
    $a \leftarrow a - \tilde{x}$, $b \leftarrow b - \tilde{y}$
    prepend $\tilde{x} + \tilde{y}\tau$ to S

Now we have set $w = 4$ in (22), we may also assume that $3 \not| k_1$ (otherwise consider the expansion of $(k_1 - 1) + k_2\tau$), so the regular form for our case is

$$k_1 + k_2\tau = \varepsilon_0 c_0 + \varepsilon_1 c_1 \tau^4 + \varepsilon_2 c_2 \tau^8 + \cdots + \varepsilon_r c_r \tau^{4r}$$

with $c_i \in C \cup C'$. Applying $\tau^{4j} = \omega^{2j} 3^{2j}$, we have a better regular form

$$k_1 + k_2\tau = \varepsilon_0 c_0 + \varepsilon_1 c'_1 3^2 + \varepsilon_2 c'_2 3^4 + \cdots + \varepsilon_r c'_r 3^{2r}$$

where $c'_j = \omega^{2j} c_j \in C \cup C'$, since $C \cup C'$ is U-invariant.

Finally, let us count the cost for this regular version of window τNAF. The pre-computation for the second set of coefficients is $2\mathbf{DBL} + 7\mathbf{ADD} + 9\mathbf{M} = 135\mathbf{M}$. So the scalar multiplication using our new method requires

$$\left(\frac{2}{4} \cdot 10 + \frac{1}{4} 16\right) \cdot \log_3 n \mathbf{M} + 94\mathbf{M} + 135\mathbf{M} < (5.68 \log_2 n + 229)\mathbf{M} \qquad (24)$$

The constant time GLV method with width 5 requires

$$\left(\frac{\log_2 n}{2} + 1\right) \cdot 7\mathbf{M} + \left(\frac{\log_2 n}{4} + 7\right) \cdot 16\mathbf{M} + 8\mathbf{M} \approx (7.5 \log_2 n + 127)\mathbf{M} \qquad (25)$$

This indicates that the constant time scalar multiplication using our new method improves $17.7\%, 19.5\%$ and 20.9% of improvements over the GLV method, when $\log_2 n = 256, 384$ and 512 respectively. When $\log_2 n$ is large, the improvement can be more than 24%. Our implementation confirms this analysis.

5 Conclusion

This paper discusses efficient computation of curves $E_b/\mathbb{F}_p$ by utilizing some nice properties from its ring $\mathbb{Z}[\omega]$ of endomorphisms, for a prime $p \equiv 1 \pmod 3$. Several efficient endomorphisms, including $\tau = 1 - \omega, \rho = 2 - \omega, 3$ and 7, are obtained. In Jacobian projective coordinates, the costs of $\tau = 1-\omega$ and $\rho = 2-\omega$ are less expensive than a point doubling and a point addition, respectively. Our computation of $3P$ and $7P$ also improve the existing results greatly.

In particular, the cost of τ is comparable to that for the Frobenius map of curves over extension fields of small characteristics. This suggests a possibility of creating a window τNAF method for the family of curves $E_b/\mathbb{F}_p$. This is achieved in the paper by creating some mathematical tools and by designing a very efficient pre-computation. To the best of our knowledge, this is the first window τNAF scalar multiplication method (where τ is a complex algebraic integer, rather than a rational integer) developed for curves over prime fields. Our pre-computation is based on a carefully designed set of coefficients, the U-invariant property of such a set has been repeated used in optimizing the performance of the window τNAF method for $E_b/\mathbb{F}_p$. In terms of the number of field multiplication used, for group orders of $256, 384$, and 512 bits, our algorithm yields a performance gain exceeding $16.7\%, 17.6\%$, and 18%, respectively, compared to the GLV method. For the purpose of dealing with simple side-channel attack, a regular window τNAF method is described, this version of scalar multiplication method shows $17.7\%, 19.5\%$ and 20.9% of improvement, for group orders of $256, 384$, and 512 bits respectively.

References

[1] Barreto, P.S.L.M., Lynn, B., and Scott, M.: Constructing elliptic curves with prescribed embedding degrees. In Security in Communication Networks 2002, Lecture Notes in Computer Science, vol. 2576, 263-273.

[2] Bernstein, D. J., Chuengsatiansup, C., and Lange, T.: Double-base scalar multiplication revisited. https://eprint.iacr.org/2017/037.

[3] Bernstein, D. J. and Lange, T.: The explicit-formulas database, 2025, https://www.hyperelliptic.org/EFD/.

[4] Blake, I. F., Murty, K. V., Xu, G.: A note on window τ-NAF algorithm, *Information Processing Letters*, 95(2005), no. 5, 496-502.

[5] Blake I. F., Murty K. V., Xu G.: Efficient algorithms for Koblitz curves over fields of characteristic three, *Journal of Discrete Algorithms*, 3(2005), 113-124.

[6] Blake, I. F., Murty, K. V., Xu, G.: Nonadjacent radix-τ expansions of integers in Euclidean imaginary quadratic number fields, *Canadian Journal of Mathematics*, 60(2008), 1267-1282.

[7] Bowe, S.: BLS12-381: New zk-SNARK elliptic curve construction, 2017, https://electriccoin.co/blog/new-snark-curve.

[8] Brown, E., Myers, B.T., Solinas, J.A.: Elliptic curves with compact parameters. Tech. Report, Centre for Applied Cryptographic Research (2001). http://www.cacr.math.uwaterloo.ca/techreports/2001/ corr2001-68.ps.

[9] Ciet M., Sica F., and Quisquater J.-J.: Analysis of the Gallant-Lambert-Vanstone method based on efficient endomorphisms: Elliptic and hyperelliptic curves. In Nyberg, K. and Heys, H. M. (eds), Selected Areas in Cryptography: SAC 2002. Lecture Notes in Computer Science, vol. 2595, 21-36.

[10] Cohen H.: A course in computational algebraic number theory. Springer, 2000.

[11] Doche, C., Icart, T., Kohel, D.R.: Efficient scalar multiplication by isogeny decompositions. Yung, M., Dodis, Y., Kiayias, A., Malkin, T. (eds.) Public Key Cryptography-PKC 2006. Lecture Notes in Computer Science, vol. 3958, 191-206.

[12] Galbraith S.D., Lin X.B., Scott M.: Endomorphisms for faster elliptic curve cryptography on a Large class of curves. Journal of Cryptology, 24(2011), 446-469.

[13] Gallant, R.P., Lambert, R.J., Vanstone, S.A.: Faster point multiplication on elliptic curves with efficient endomorphisms. In: Kilian, J., (ed.) Advances in Cryptology - CRYPTO 2001. Lecture Notes in Computer Science, vol. 2139, pp. 190-200. Springer, (2001).

[14] Hankerson, D., Menezes, A., Vanstone, S.: Guide to elliptic curve cryptography. New York, NY, USA: Springer-Verlag, 2004.

[15] Hu, Z., Longa, P., Xu, M.: Implementing the 4-dimensional GLV method on GLS elliptic curves with j-invariant 0. Des., Codes Cryptography, 63(2012) , 331-343.

[16] Joye, M., Tunstall, M.: Exponent recoding and regular exponentiation algorithms. In: Preneel, B. (ed.) Advances in Cryptology-AFRICACRYPT 2009. Lecture Notes in Computer Science, vol. 5580, 334-349. Springer, Heidelberg (2009)

[17] Koblitz, N.: CM-curves with good cryptographic properties. In: Feigenbaum, J. (eds) Advances in Cryptology-CRYPTO 1991. Lecture Notes in Computer Science, vol. 576, 279-287. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-46766-1_22

[18] Koblitz, N.: An elliptic curves implementation of the finite field digital signature algorithm. In Krawczyk, H. (eds) Advances in Cryptology-CRYPTO'98. Lecture Notes in Computer Science, vol. 1462, 327-337. Springer-Verlag Berlin Heidelberg, (1998).

[19] Koblitz, N.: Algebraic aspects of cryptography, Springer, 1999.

[20] Longa P. and Miri A.: Fast and flexible elliptic curve point arithmetic over prime fields, *IEEE Transactions on Computers*, 57(2008) , 289-302.

[21] Longa, P., Sica, F.: Four-dimensional gallant-lambert-vanstone scalar multiplication. Advances in Cryptology-ASIACRYPT 2012. Beijing, China: Springer, 718-739, 2012.

[22] Longa, P., Sica, F.: Four-dimensional gallant-lambert-vanstone scalar multiplication. Journal of Cryptology, 27(2014), 248-283.

[23] Oliveira, T., Aranha, D.F., López, J., Rodríguez-Henríquez, F.: Fast point multiplication algorithms for binary elliptic curves with and without precomputation. SAC 2014, LNCS, vol. 8781, 324-344. Springer, Heidelberg, 2014.

[24] Oliveira, T., López, J., Cervantes-Vázquez, D., Rodríguez-Henríquez, F.: Software implementation of Koblitz curves over quadratic fields. Journal of Cryptology, 32(2019), 867-894.

[25] Rajwade, A. R.: On rational primes p congruent to 1 (mod 3 or 5), Proc. Cambridge Philos. Sot. 66 (1969), 61-70.

[26] Smith, B.: Easy scalar decompositions for efficient scalar multiplication on elliptic curves and genus 2 Jacobians, https://eprint.iacr.org/2013/672.pdf.

[27] Smith, B.: Families of fast elliptic curves from Q-curves. In Sako, K. and Sarkar, P. (eds) Advances in Cryptology - ASIACRYPT 2013, Part I. Lecture Notes in Computer Science, vol. 8269, 61-78. https://doi.org/10.1007/978-3-642-42033-7_4.

[28] Solinas, J.: Efficient arithmetic on Koblitz curves, Des., Codes Cryptography, 19(2000) , 195-249.

[29] Takagi, T., Yen, S.-M. and Wu, B.-C.: Radix-r non-adjacent form, Information Security Conference – ISC 2004. Lecture Notes in Computer Science, vol. 3225, pp. 99-110. Springer-Verlag, Berlin, 2004.

[30] Trost, W. and Xu, G.: On the optimal pre-computation of window τNAF for Koblitz curves. IEEE Transactions on Computers. Vol. 65, No. 9. 2016, 2918-2924.

[31] Wu, H. and Xu, G.: On Koblitz curves over prime fields. Journal of Cryptologic Research, 2024, Vol. 11, Issue (5), 1152-1159. DOI: 10.13868/j.cnki.jcr.000735

[32] Yu, W. and Xu, G.: Pre-computation scheme of window τNAF for Koblitz curves revisited. In Canteaut, A. and Standaert, F.-X. (eds) Advances in Cryptology–EUROCRYPT 2021, Part II. Springer Cham, Lecture Notes in Computer Science, vol. 12697, 187-218. DOI: 10.1007/978-3-030-77886-6_7

[33] SP 800-186 recommendations for discrete logarithm-based cryptography: elliptic curve domain parameters, *https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-186.pdf*,2023.

[34] SEC 2: recommended elliptic curve domain parameters, *https://www.secg.org/sec2-v2.pdf*, 2010.